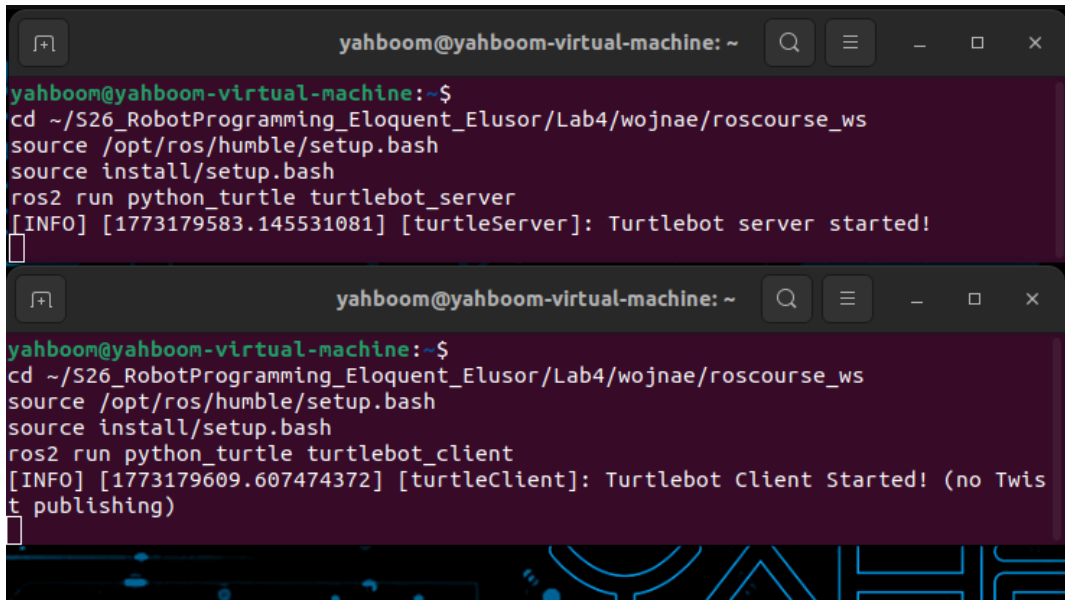


Lab 4 Deliverable

Task 1: Step 17

Screenshots of terminals:

turtlebot_server



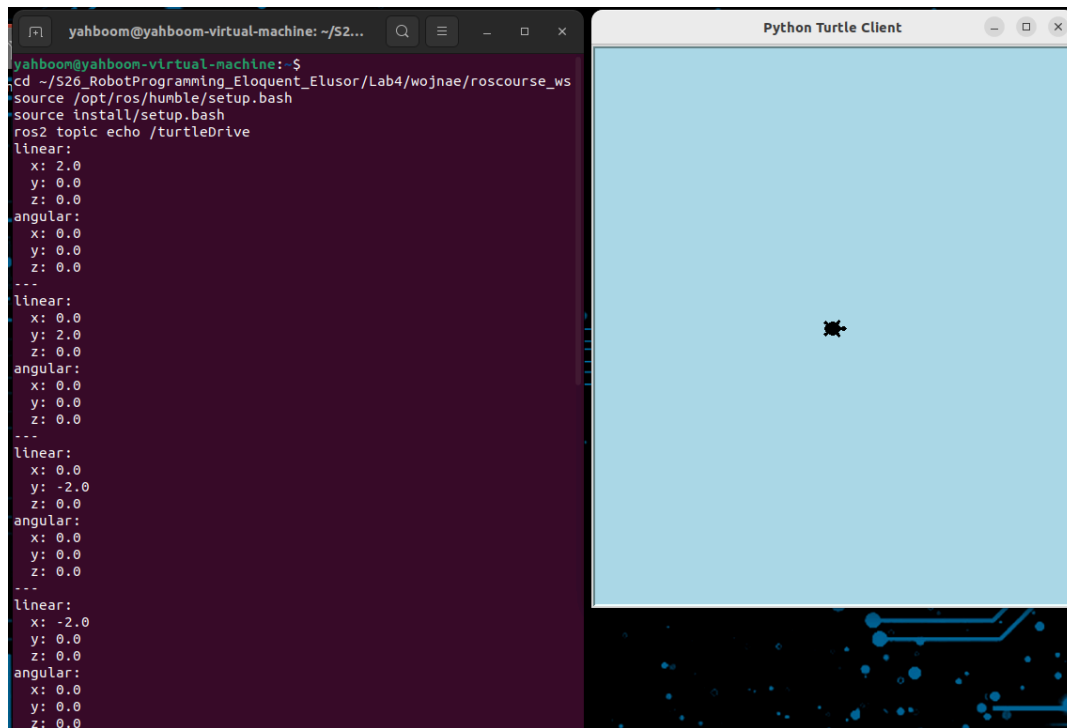
The image shows two terminal windows from a virtual machine named 'yahboom@yahboom-virtual-machine'. The top terminal window shows the execution of the 'turtlebot_server' command, which outputs '[INFO] [1773179583.145531081] [turtleServer]: Turtlebot server started!'. The bottom terminal window shows the execution of the 'turtlebot_client' command, which outputs '[INFO] [1773179609.607474372] [turtleClient]: Turtlebot Client Started! (no Twist publishing)'. Both windows show the same directory path: '~/S26_RobotProgramming_Eloquent_Elusor/Lab4/wojnae/roscourse_ws'.

```
yahboom@yahboom-virtual-machine: ~  
yahboom@yahboom-virtual-machine:~$  
cd ~/S26_RobotProgramming_Eloquent_Elusor/Lab4/wojnae/roscourse_ws  
source /opt/ros/humble/setup.bash  
source install/setup.bash  
ros2 run python_turtle turtlebot_server  
[INFO] [1773179583.145531081] [turtleServer]: Turtlebot server started!  
yahboom@yahboom-virtual-machine:~$  
cd ~/S26_RobotProgramming_Eloquent_Elusor/Lab4/wojnae/roscourse_ws  
source /opt/ros/humble/setup.bash  
source install/setup.bash  
ros2 run python_turtle turtlebot_client  
[INFO] [1773179609.607474372] [turtleClient]: Turtlebot Client Started! (no Twist publishing)
```

turtlebot_client

```
yahboom@yahboom-virtual-machine: ~  
yahboom@yahboom-virtual-machine:~$ cd ~/S26_RobotProgramming_Eloquent_Elusor/Lab4/wojnae/ros  
course_ws  
source /opt/ros/humble/setup.bash  
source install/setup.bash  
  
ros2 run teleop_twist_keyboard teleop_twist_keyboard \  
--ros-args \  
--remap cmd_vel:=/turtleDrive \  
--remap /turtlebot/cmd_vel:=/turtleDrive \  
-p stamped:=false \  
-p speed:=2.0 \  
-p turn:=0.0  
  
This node takes keypresses from the keyboard and publishes them  
as Twist/TwistStamped messages. It works best with a US keyboard layout.  
-----  
Moving around:  
  u    i    o  
  j    k    l  
  m    ,    .  
  
For Holonomic mode (strafing), hold down the shift key:  
-----  
  U    I    O  
  J    K    L  
  M    <    >  
  
t : up (+z)  
b : down (-z)  
  
anything else : stop  
  
q/z : increase/decrease max speeds by 10%  
w/x : increase/decrease only linear speed by 10%  
e/c : increase/decrease only angular speed by 10%  
  
CTRL-C to quit  
  
currently:      speed 2.00      turn 0.00
```

echo showing direction turtle is moving in when pressing i, s, l, etc. and turtle screen



Task 2: Step 14

Modified turtlebot_client.py:

```
#!/usr/bin/env python3  
====  
(Lab 4 - Task 2)  
Purpose:  
- Add ROS 2 parameters to set the turtle's initial color and pen size on startup.  
- Ensure the server respects our chosen color by calling the /set_color service at init.  
- Keep this client as a "display-only" node (no Twist publishing) so teleop controls motion.  
====  
  
import math  
import rclpy  
from rclpy.node import Node  
  
# New imports required by Task 2  
from rclpy.parameter import Parameter  
from rcl_interfaces.msg import ParameterDescriptor  
  
# Turtle graphics  
import turtle  
  
# Messages / services  
from geometry_msgs.msg import Pose  
from turtle_interfaces.msg import TurtleMsg  
from turtle_interfaces.srv import SetColor # service provided by /turtleServer  
  
# Basic named-colors -> RGB map (only used if server expects r/g/b fields)  
_NAMED_RGB = {  
    'black': (0, 0, 0),  
    'white': (255, 255, 255),  
    'red': (255, 0, 0),  
    'green': (0, 255, 0),  
    'blue': (0, 0, 255),  
    'cyan': (0, 255, 255),  
    'magenta': (255, 0, 255),
```

```

'yellow': (255, 255, 0),
'orange': (255, 165, 0),
'purple': (128, 0, 128),
'pink': (255, 105, 180),
'gray': (128, 128, 128),
}

```

```

def _color_to_rgb(color_str: str):

```

```

    """
    Accept a color name (e.g., 'orange') or '#RRGGBB' and return integer RGB.
    Falls back to magenta if value is unrecognized.
    """

```

```

    s = (color_str or "").strip().lower()
    if s.startswith('#') and len(s) == 7:
        try:
            r = int(s[1:3], 16)
            g = int(s[3:5], 16)
            b = int(s[5:7], 16)
            return (r, g, b)
        except ValueError:
            pass
    if s in _NAMED_RGB:
        return _NAMED_RGB[s]
    # fallback
    return _NAMED_RGB['magenta']

```

```

class TurtleClient(Node):

```

```

    def __init__(self):
        super().__init__('turtleClient')

        # Parameters (Task 2)
        # turtleColor: color to apply on startup. Examples: 'orange', 'magenta', '#00FF00'
        color_desc = ParameterDescriptor(description='Initial turtle pen/body color')
        self.declare_parameter('turtleColor', 'magenta', color_desc)

        # penSize: pen thickness in pixels
        pen_desc = ParameterDescriptor(description='Initial turtle pen size in pixels')
        self.declare_parameter('penSize', 2, pen_desc)

        # Read parameter values
        self.turtle_color: str = self.get_parameter('turtleColor').get_parameter_value().string_value
        self.pen_size: int = int(self.get_parameter('penSize').get_parameter_value().integer_value)

        # Display / Turtle Setup
        self.screen = turtle.Screen()
        self.screen.title("Python Turtle Client")
        self.screen.bgcolor("lightblue")

        # Optional: expand the drawing world; uncomment if needed
        # self.screen.setworldcoordinates(-300, -300, 300, 300)

        # Speed up drawing; manual frame updates
        self.screen.tracer(0)

        self.turtle_display = turtle.Turtle(visible=True)
        self.turtle_display.shape("turtle")
        # Apply parameterized visual settings immediately
        self.turtle_display.pencolor(self.turtle_color)
        self.turtle_display.color(self.turtle_color) # outline/body color
        self.turtle_display.pensize(self.pen_size)
        self.turtle_display.penup() # pen state will switch in update if server color is set

        # We keep a local copy of the last received TurtleMsg
        self.turtle = TurtleMsg()

        # Subscriptions
        self.turtle_sub = self.create_subscription(
            TurtleMsg, 'turtleState', self.turtle_callback, 10
        )

        # Ensure server uses our color (Task 2)
        # The server publishes a blank color initially; if we don't call /set_color,
        # update_display() will see 'None' and keep pen up. So we set it now.
        self.color_cli = self.create_client(SetColor, '/set_color')
        while not self.color_cli.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('Color service not available, waiting...')

        # Create request. Try both styles:
        # - string field 'color' (some labs use this)
        # - int fields r,g,b (our local IDL supports this layout too)

```

```

self.color_req = SetColor.Request()
if hasattr(self.color_req, 'color'):
    # String color interface
    self.color_req.color = self.turtle_color
else:
    # RGB interface
    r, g, b = _color_to_rgb(self.turtle_color)
    # Keep values in 0..255
    self.color_req.r = int(r)
    self.color_req.g = int(g)
    self.color_req.b = int(b)

# Fire and forget; log when done
self.service_future = self.color_cli.call_async(self.color_req)
self.service_future.add_done_callback(self._on_color_response)

# Update display at ~30 Hz
self.timer = self.create_timer(1.0 / 30.0, self.update_display)

self.get_logger().info(
    f"Turtlebot Client Started! (turtleColor='{self.turtle_color}', penSize={self.pen_size})"
)

# Callbacks & utilities

def _on_color_response(self, future):
    try:
        resp = future.result()
        # Both request variants have 'success' and 'message' in our IDL—ignore if not found
        ok = getattr(resp, 'success', True)
        msg = getattr(resp, 'message', "")
        if ok:
            self.get_logger().info(f"/set_color acknowledged: {msg}")
        else:
            self.get_logger().warn(f"/set_color failed: {msg}")
    except Exception as e:
        self.get_logger().error(f"/set_color call failed: {e}")

def turtle_callback(self, msg: TurtleMsg):
    self.turtle = msg

@staticmethod
def rpy_from_quat(x, y, z, w):
    """Convert quaternion to roll, pitch, yaw."""
    srCP = 2.0 * (w * x + y * z)
    crCP = 1.0 - 2.0 * (x * x + y * y)
    roll = math.atan2(srCP, crCP)

    sp = 2.0 * (w * y - z * x)
    if abs(sp) >= 1.0:
        pitch = math.copysign(math.pi / 2.0, sp)
    else:
        pitch = math.asin(sp)

    syCP = 2.0 * (w * z + x * y)
    cyCP = 1.0 - 2.0 * (y * y + z * z)
    yaw = math.atan2(syCP, cyCP)
    return roll, pitch, yaw

def update_display(self):
    """
    Render pose + pen color from latest TurtleMsg.
    If server color is blank ('None'), keep pen up; after /set_color returns,
    color will be non-blank and we draw the path.
    """
    color = (self.turtle.color or "").strip()
    if color and color.lower() != 'none':
        self.turtle_display.pendown()
        self.turtle_display.pencolor(color)
    else:
        # While server color is None, don't draw lines
        self.turtle_display.penup()

    px = self.turtle.turtle_pose.position.x
    py = self.turtle.turtle_pose.position.y
    ox = self.turtle.turtle_pose.orientation.x
    oy = self.turtle.turtle_pose.orientation.y
    oz = self.turtle.turtle_pose.orientation.z
    ow = self.turtle.turtle_pose.orientation.w

    self.turtle_display.setpos(px, py)

```

```

        yaw = self.rpy_from_quat(ox, oy, oz, ow)
        self.turtle_display.seth(math.degrees(yaw))
        self.screen.update()

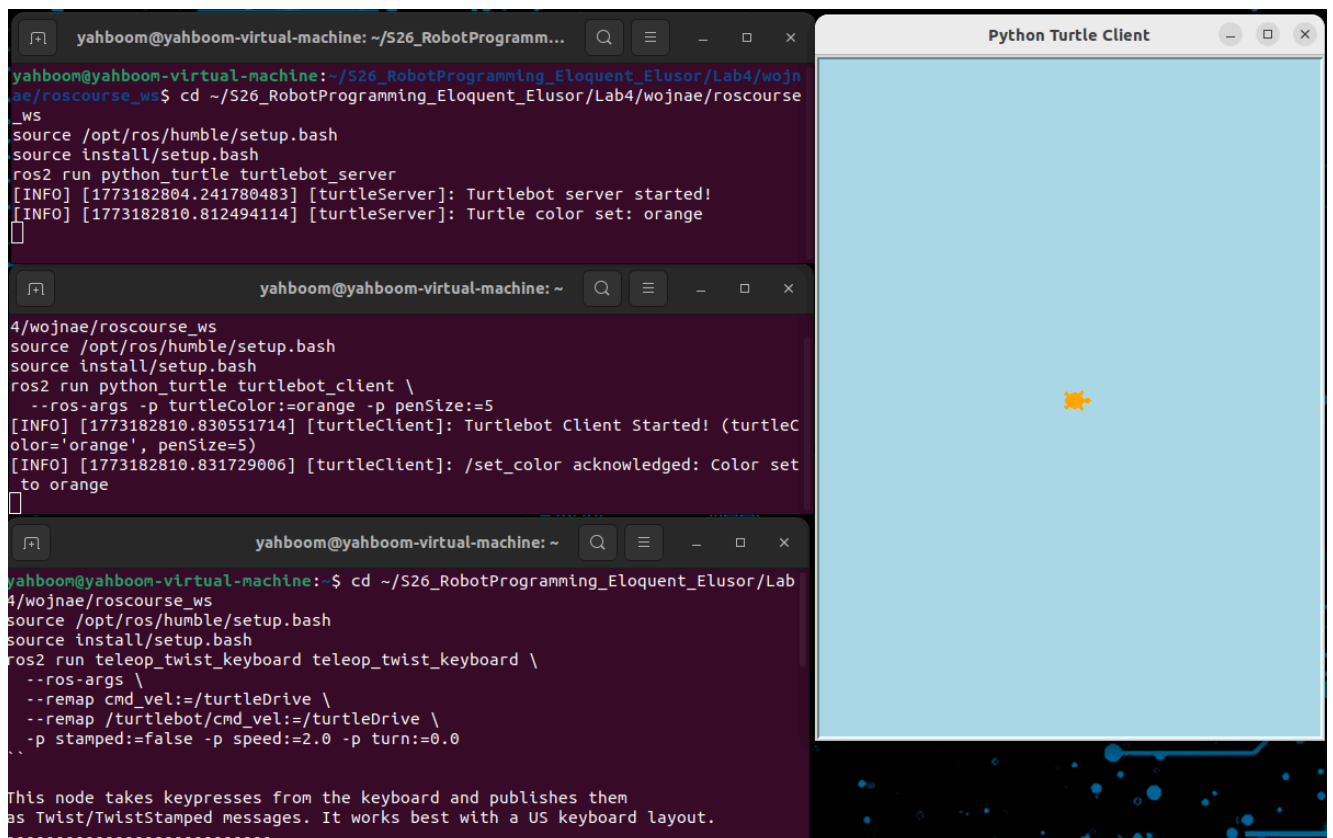
def main(args=None):
    rclpy.init(args=args)
    node = TurtleClient()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.get_logger().info('Shutting down Turtlebot Client.')
        node.destroy_node()
        rclpy.shutdown()

if __name__ == '__main__':
    main()

```

Task 2: Step 13

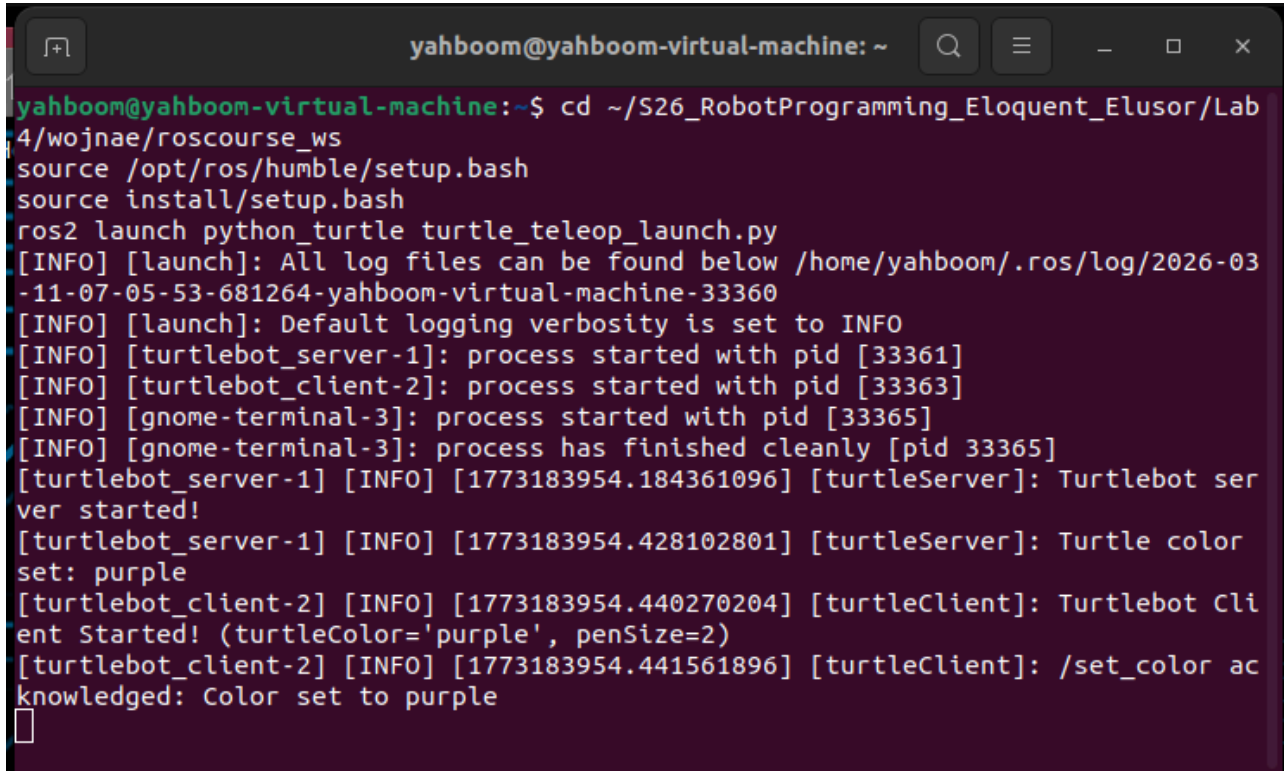
Screenshot of turtle (different color than the default parameter value):



Task 3: Step 8

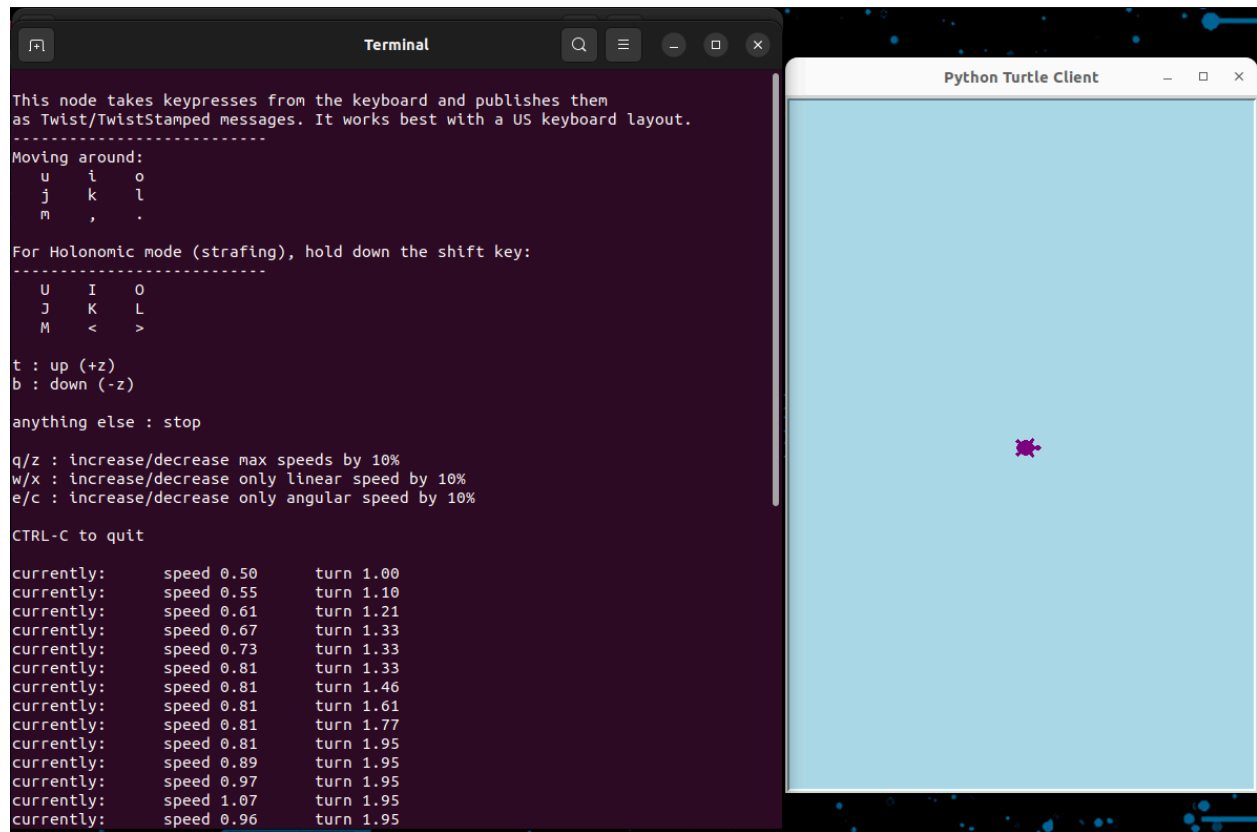
Screenshot of terminals:

turtle_teleop_launch.py

A terminal window titled 'yahboom@yahboom-virtual-machine: ~' with standard window controls. The terminal shows the execution of a ROS2 launch script. The user navigates to a directory and runs 'ros2 launch python_turtle turtle_teleop_launch.py'. The output displays various log messages from the launch system and the turtlebot server/client, including process start times, PIDs, and confirmation that the turtle color is set to purple.

```
yahboom@yahboom-virtual-machine:~$ cd ~/S26_RobotProgramming_Eloquent_Elusor/Lab
4/wojnae/roscourse_ws
source /opt/ros/humble/setup.bash
source install/setup.bash
ros2 launch python_turtle turtle_teleop_launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-03-11-07-05-53-681264-yahboom-virtual-machine-33360
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [turtlebot_server-1]: process started with pid [33361]
[INFO] [turtlebot_client-2]: process started with pid [33363]
[INFO] [gnome-terminal-3]: process started with pid [33365]
[INFO] [gnome-terminal-3]: process has finished cleanly [pid 33365]
[turtlebot_server-1] [INFO] [1773183954.184361096] [turtleServer]: Turtlebot server started!
[turtlebot_server-1] [INFO] [1773183954.428102801] [turtleServer]: Turtle color set: purple
[turtlebot_client-2] [INFO] [1773183954.440270204] [turtleClient]: Turtlebot Client Started! (turtleColor='purple', penSize=2)
[turtlebot_client-2] [INFO] [1773183954.441561896] [turtleClient]: /set_color acknowledged: Color set to purple
█
```

Pop-up terminal and turtle screen



Task 4:

Screenshot of window with multiple turtles that have moved with different colors:

